

MockGen: Realistic Offline Mock Data for Enterprise Applications, Packaged for the Editor

Christos Koutsiaris

Abstract—Developers of enterprise applications need screens full of realistic data long before any backend exists. Hand-written mock data is expensive and goes stale; random generators produce values that look broken on screen. MockGen generates complete, relationally consistent mock datasets for an application directly from its service definition, fully offline and deterministically. Every column is resolved by a cascade: declared catalogs first, then a recognised semantic role, then a small local language model, then a type-correct fallback that can never decline. This paper describes the whole system as it ships — the schema representation and its adapters, the four tiers, the relational passes that keep keys unique and references resolvable, the budget governance that makes a small model useful on weak hardware, and the packaging and editor integration that put it in front of developers. It also reports the part that changed most recently: role recognition is now a learned classifier trained on 23,741 fields labelled by a panel of three large language models that see only structural metadata. On 131 real service definitions the pipeline generates 11,529 fields with 0 failures and binds 44.79% of them to a semantic provider; with recognition disabled the same corpus binds 22.93% and 12 services fail outright. On 41 services held out from training, the classifier’s own decisions are correct 96.2% of the time across 44 roles. We also report a component that did not work — a field-to-value relevance verifier — and the shipping decision we made about it.

Index Terms—mock data, test data generation, service metadata, schema graph, OData, offline inference, small language models, semantic role classification, relational data generation

I. INTRODUCTION

A FRONTEND developer building an enterprise application starts from a service definition: a machine-readable contract that names entities, columns, types, widths, and relationships. The application must show data immediately, before any backend exists. The usual options disappoint. Hand-written mock data is slow to produce and drifts out of sync with the schema. Generic random generators fill a city column with random words, so every screen looks broken. Recording real responses needs access, permissions, and the removal of sensitive values.

Enterprise schemas make this harder. One service routinely declares dozens of entities and hundreds of columns with exact widths and precisions. Columns carry protocol artifacts — draft flags, edit shadows — that are not business data. The schema’s own annotations encode meaning a generator should honour. Mistakes are visible: a too-long value truncates on screen, a code that does not match its text reads as a bug, and one empty child table makes the application look defective.

C. Koutsiaris is with SAP P&E – Cloud ERP – UX Foundation (e-mail: christos.koutsiaris@sap.com).

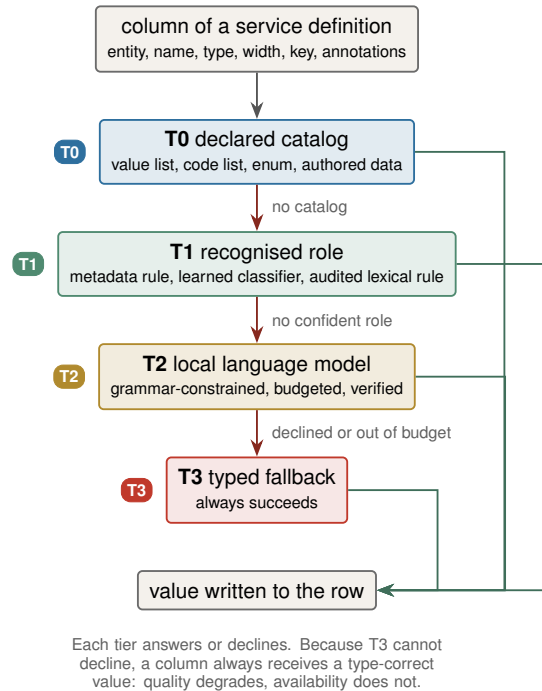


Fig. 1. The resolution cascade for a single column. Declared catalogs beat recognition, recognition beats the language model, and the typed fallback cannot refuse. Takeaway: the never-empty promise is structural, not aspirational.

MockGen reads the application’s service definition and generates a complete dataset for it: offline, deterministic, in seconds. Five rules govern the design.

- 1) **Any application, cold.** An application MockGen has never seen works without configuration.
- 2) **No empty screens.** Lists, object pages, and child tables always have rows.
- 3) **It must look real at a glance.** Codes match their texts, child rows belong to their parents, one record tells one story.
- 4) **No internal failure blanks the application.** A missing model or a rejected value costs realism, never availability.
- 5) **Determinism.** The same schema, seed, and artifacts produce byte-identical data.

The architecture that delivers this is a *tier cascade* (Fig. 1): for each column, the first tier that can produce a constraint-valid value wins, and the last tier cannot decline.

Contributions. (i) A schema intermediate representation (IR) and adapters that let one engine serve several service formats (Section III). (ii) The four tiers and their mechanics (Sections IV–X). (iii) A learned role recogniser trained on

machine-adjudicated labels, with a routing policy that decides per role whether the model may speak (Sections VII–VIII). (iv) The relational passes, determinism, and caching (Sections XI–XII). (v) Packaging, qualification, and editor integration (Section XIII). (vi) A measured characterisation of the shipped system, including a negative result (Sections XIV–XV).

II. RELATED WORK

Random and property-based generators produce type-correct values and are excellent for testing invariants, but they have no notion of what a column means, so their output looks wrong on a screen.

Database synthesisers learn distributions from existing data. They need that data; our setting has none, because the backend does not exist yet.

Schema matching aligns two schemas with each other. Our problem aligns one schema against a fixed vocabulary of generation roles. The output is not a correspondence but a decision about which value generator to call, and the cost of being wrong differs: a bad match is reviewed by a human, a bad role silently writes a city into an order number.

Hosted language models would classify columns well and are unavailable at run time: the generator must work with no network, in a locked-down development space, deterministically. We therefore use large models offline, once, to produce labels, and ship a small model that reproduces their decisions locally.

III. SCHEMA IR AND ADAPTERS

Everything downstream reads one internal shape: a schema graph. Adapters translate each supported input format into it, so the tiers never learn protocol dialects.

The graph holds entities with their entity-set names; properties with primitive type, width, precision, scale, key and nullability flags, declared labels, data elements, and descriptions; relationships with their column mappings; and annotation-derived links such as value lists, text associations, and currency or unit companions.

Two adapters ship today. The Extensible Markup Language metadata format (EDMX) used by Open Data Protocol (OData) services is parsed structurally. Core Data Services (CDS) documents are parsed from their compiled notation. The field context the recogniser reads is derived from this graph, and it is the same object at training time and at run time — a property enforced by fingerprints (Section XII).

IV. THE CASCADE AT A GLANCE

For every column of every row, the tiers are attempted in order.

T0, declared catalogs. If the schema declares the allowed values — a value list, a code list, an enumeration — or the developer supplied authored rows, the value comes from there. Nothing beats the schema’s own statement of truth.

T1, recognised roles. Otherwise the system tries to recognise what the column means and draws from a curated bank for that meaning. Recognition has three sources: annotation-driven metadata rules, the learned classifier, and audited name-based rules.

T2, a local language model. If no role is recognised and the column is visible text that would otherwise read as a placeholder, a small quantized model proposes a value under a grammar and a time budget.

T3, the guaranteed floor. Otherwise a typed generator emits a value that respects type, width, precision, and nullability. It cannot decline.

The cascade is per column, not per table, so one entity commonly draws from three tiers at once.

V. T0: AUTHORITATIVE CATALOGS

T0 is the cheapest and most trustworthy tier. A property annotated with a value list points at a code entity; the generator reads that entity’s rows if present, and otherwise generates the code table first and draws from it, so a code column and its text column agree by construction.

Authored data is also T0. A developer with real sample rows drops them in, and those values win over anything the generator would produce, while the generator still fills the columns the sample omits.

VI. T1: RECOGNISED ROLES

T1 answers the question this system is really about: what does this column mean? A role is not a free-form string. Each registered role declares the primitive types it accepts, whether it may sit on a key, which provider generates its values, which validator checks them, the coherence group it belongs to, and the confidence needed to route it. Roles are grouped into families — status, identity, location, finance, temporal, narrative, contact, organisation, measure — and families matter for both support floors and error severity. Assigning a city where the label says postal code is an error inside the location family; assigning a city where the label says status crosses families and counts as critical.

A. Three sources, one arbitrator

Metadata rules fire when an annotation, data element, or explicit type determines the role outright. Their decisions are decisive: the classifier is not consulted.

The learned classifier handles everything else. It reads only the field context and never sees values.

Audited name-based rules remain where a name is genuinely conclusive. Each rule was checked against adjudicated labels; the eight that contradicted a label with a different positive role were switched off, while rules whose only misses were abstentions were kept, because they carry generation value — coherent key domains — that labels do not measure.

The arbitrator combines them. Metadata wins outright. If the classifier abstains, a surviving lexical rule may fire. If the classifier routes and a lexical rule disagrees, the classifier wins only when its prediction set is a singleton and its confidence clears a stricter conflict threshold; otherwise both are discarded, because two disagreeing weak signals are worse than none. Compatibility is enforced last: a role its registry entry forbids on keys cannot land on a key, and a role incompatible with the column’s type is rejected regardless of source.

B. Framework columns never route

Draft-administration and action-control columns are framework bookkeeping, not business data. They abstain unconditionally, because inventing a plausible business value for them is worse than leaving them to the typed floor.

VII. LEARNING TO RECOGNISE A COLUMN

A. Labels without human annotators

Hand-labelling enterprise columns needs domain experts and time. We instead use a panel of three large language models from two vendors: Claude Opus 4.8, Claude Sonnet 4.5, and GPT-5.6-terra. Each judge sees one field’s structural metadata and a frozen written guideline, and returns a registered role or the abstention label `unknown`. Judges are independent: none sees another’s answer, any model prediction, or any data value. A field is labelled when at least two of three agree; disagreements are recorded and excluded from every partition rather than resolved by a tie-break, which would manufacture agreement.

six rounds produced 23,741 labelled fields. Agreement, measured as Fleiss’ kappa, ran from 0.87 on the first round down to 0.72 on a round aimed deliberately at the hardest cases — columns named `name`, `text`, `title` — and recovered to 0.81 after we clarified the guideline for exactly those cases and re-judged them.

Every label carries its provenance: method *machine panel consensus*, the judges and their prompt fingerprints, and the flag `humanVerified` set to `false`. We do not claim these labels match expert human labels. We claim they are consistent, reproducible, and honestly described.

B. Partitions and leakage

Services are grouped into families by the source registry, and a family belongs to exactly one partition, so a sealed service shares no lineage with anything the model learned from. Training uses 12,375 rows from 119 service groups; calibration uses 4,991 rows from 39 groups; the sealed set is 4,317 fields across 41 services and 21 domains.

Two subtler leaks are closed in the same step. Two fields can serialize to identical text and carry different labels; those contexts are dropped everywhere, because a context with two labels silently caps accuracy. And a context appearing in two partitions has its lower-priority copy dropped. Neither rule relabels anything; both only remove.

C. Writing the field as language

The classifier embeds a text rendering of the field context with a frozen sentence encoder, then classifies the embedding (Fig. 2). The rendering mattered more than anything else we changed.

The pilot rendered a delimited record: `v3|BookingStatus_code|string|TravelBooking|`. That is a sensible machine format and a poor input for an encoder trained on prose. The shipped rendering is a sentence: `Booking Status code (string) in Travel Booking; data element BOOK STATUS;`

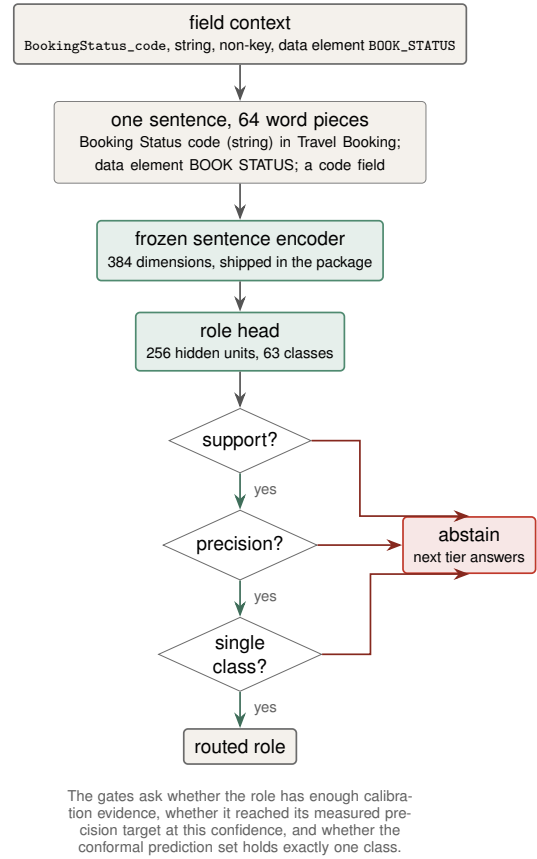


Fig. 2. How one column becomes a routed role. The field context is rendered as a sentence, embedded by the frozen encoder, and scored by a small head; three gates can each send the decision back to abstention. Takeaway: routing is a decision about trust, not simply the top-scoring class.

a code field. Identifiers are split into words; structural facts are stated in words; and verbose material that consumes the 64-word-piece budget without carrying role evidence — neighbour lists, facet objects, annotation arrays — is dropped from the text while remaining available to the metadata tier. With the same encoder, labels, and head, calibration accuracy rose from 0.63 to 0.84. We had been preparing to fine-tune the encoder; the measurement showed the encoder was never the bottleneck.

The final clause names the identifier’s head noun. Mean pooling cannot weigh the last word of an identifier, yet that word decides meaning: `BookingStatus` is a status, `BookingStatusName` is a name. Stating it raised status routing precision on calibration from 45/56 to 91/100.

D. Head capacity

The pilot head was a single linear layer over the embedding. With 63 classes and closely related roles, that is too little capacity. One hidden layer of 256 rectified-linear units on the same embeddings raised calibration accuracy from 0.62 to 0.84, and the share of role-bearing calibration fields the system is willing to route from 29% to 74%. The shipped head is a multilayer perceptron (MLP) of $384 \times 256 \times 63$, adding 3.4 MiB to the package and running in microseconds per field.

TABLE I

THE TWO DESIGN CHANGES, MEASURED ON IDENTICAL LABELS AND THE SAME FROZEN ENCODER. ROUTED SHARE IS THE FRACTION OF ROLE-BEARING CALIBRATION FIELDS THE SYSTEM IS WILLING TO ROUTE.

Configuration	Cal. accuracy	Routed share
Delimited text, linear head	0.63	—
Sentence text, linear head	0.84	29%
Sentence text, 256-unit head	0.84	74%

VIII. WHEN THE CLASSIFIER MAY SPEAK

A decision routes only if it passes three gates.

Calibration support. The role needs at least 5 correct calibration examples and its family at least 10. A role the system cannot measure is one it must not use.

Measured precision. For each role we replay the runtime acceptance rule on the calibration partition and raise that role’s threshold — never lower it — to the lowest confidence at which its routed decisions reach 0.95 precision with enough decisions to measure. A role that cannot reach the target at any threshold does not route.

Conformal set size. Calibration yields a quantile targeting 0.900 coverage; measured coverage is 0.900. A decision whose prediction set holds more than one class abstains, whatever its confidence.

Roles failing these gates stay in the model as *auxiliary* classes with a route threshold above one: trained, never selected. This matters more than it sounds. When we instead removed weak classes from training, status-text columns had nowhere to go and were misrouted as status codes. Of 63 trained classes, 44 route and 18 are auxiliary. Coverage therefore grows with evidence rather than with code changes: more labels promote a role from auxiliary to routable without touching the generator.

IX. T2: A BUDGETED LOCAL LANGUAGE MODEL

When no role is recognised and the column is visible text, a small quantized model proposes values. Three mechanisms keep it useful on two virtual central processing units (vCPUs).

Grammar constraint. Decoding is constrained to a grammar derived from the request, so output parses as rows with the expected columns and cannot drift into prose.

Budget governance. Generation runs under a wall-clock budget per entity. When the budget expires the tier yields and the cascade continues; the run never stalls waiting for a model.

Verification. A separate verifier judges whether a proposed value suits its field before it is written. Section XV reports what happened when we measured it.

X. T3: THE GUARANTEED FLOOR

T3 cannot decline. It emits values respecting the declared type, width, precision, scale, and nullability, staying unique where uniqueness is required. Its values are plainly generic — this is the tier that produces `String 1` — and that is the point: it turns “no empty screens” from a wish into a property.

XI. KEYS, REFERENCES, AND CACHING

Realism at the row level is not enough; a dataset must hold together.

Keys. Key columns are generated first and kept unique within their entity, respecting width limits so a key never truncates.

References. A relationship’s child rows draw their foreign key values from the parent rows that were actually generated, so every reference resolves. Child tables are populated for the parents the user will open, so object pages are not empty.

Coherence. Columns in a coherence group are generated together: a city with its country and postal code, an amount with its currency, a quantity with its unit. A second pass recomputes derived display values after the domains they depend on are final.

Caching. Generated datasets are cached under a key including the schema, the seed, the row counts, and the fingerprints of every model artifact. Changing any of them changes the key, so a stale dataset is never served after a model swap.

XII. DETERMINISM AND REPRODUCIBILITY

Determinism is a product promise. Generation is seeded; the classifier performs no sampling; the text rendering is a pure function of the field context; and the tokenizer budget is fixed, so a field that overflows it overflows identically during training export and at run time.

Reproducibility of the models is enforced by fingerprints. The role head records the checksum of the encoder it was trained against, the checksum of the vocabulary, a fingerprint of the text serializer, and a fingerprint of the role registry. At load time the runtime recomputes all four and refuses a head whose fingerprints do not match the code it is running inside. Changing the role vocabulary or the serialization therefore invalidates the packaged head instead of silently degrading it.

XIII. PACKAGING, QUALIFICATION, AND THE EDITOR

A. *Nothing is qualified by hand*

A trained head is unqualified until a tool re-runs the full runtime contract against it, recomputes the sealed dataset fingerprint, verifies the label review record, checks that no developer path or unresolved marker appears in the artifact, and only then writes the qualified status. The packaged manifest is regenerated from file content, so a hand-edited model file fails the release check.

B. *What ships where*

The editor extension carries no models (Fig. 3). It pins the package by name, version, application programming interface (API) version, integrity hash, and distribution hash, and resolves that pin once into a shared module cache. The package carries the engine and the model component: encoder 21.9 MiB, role head 3.4 MiB, vocabulary 0.2 MiB, relevance head 0.1 MiB, and the generation model 157.3 MiB. The published archive is 128 MB against a 200 MB registry limit. Verification is defence in depth: the archive is checked by integrity hash on download, each model file by checksum against the manifest, the manifest’s revision is recomputed from content, and cached executable code is re-checked before import, with a modified file quarantined rather than used.

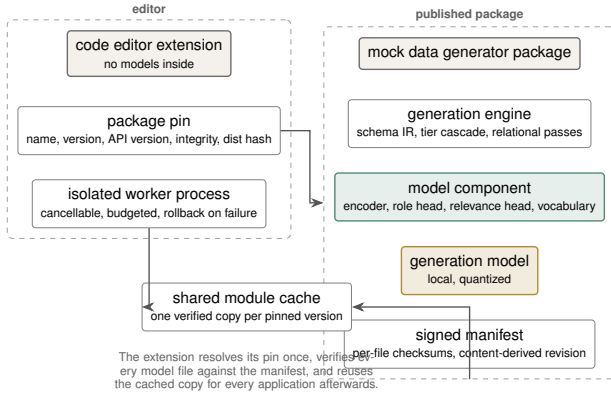


Fig. 3. What ships where. The extension holds only a pin; the package holds the engine and the models; a shared cache holds one verified copy per pinned version. Takeaway: models are distributed once and verified on every load, not bundled into every extension update.

C. Availability without a stall

One defect here is worth recording because it is easy to reproduce. The editor decided whether to offer the action by calling the same routine that installs the package, under a 60-second budget, while a first-time install of several hundred megabytes cannot finish in that time. The menu entry therefore never appeared on a fresh machine. The availability check now verifies platform, runtime version, and pin, and reports a valid but uninstalled pin as available with a pending-install flag; the install happens on first generation, inside the 300-second generation budget.

XIV. RESULTS

A. Corpus coverage

We run the whole pipeline over every checksum-verified service in the source registry whose format this package parses directly: 131 services of 473 registered, covering 1,345 entities and 11,529 fields, at two rows per entity. All 131 generate with 0 failures, including a service with 897 fields.

Of those fields, 44.79% bind to a semantic provider: the classifier contributes 29.01%, metadata rules 5.96%, and audited lexical rules 11.85%. The remaining 53.18% abstain and are served by the lower tiers.

Running the same corpus with recognition disabled isolates what the learned tier adds (Fig. 4). Semantic binding falls from 44.79% to 22.93%, and — the result we did not anticipate — 12 of 131 services fail outright, because without a recognised role some code columns cannot satisfy their own semantic validation. Recognition is therefore not only a realism feature but a robustness one.

B. Recognition quality on unseen services

Corpus shares say how often the classifier speaks; they do not say whether it is right. For that we use the sealed set, which is family-disjoint from training and calibration and is scored through the real runtime path rather than a notebook metric.

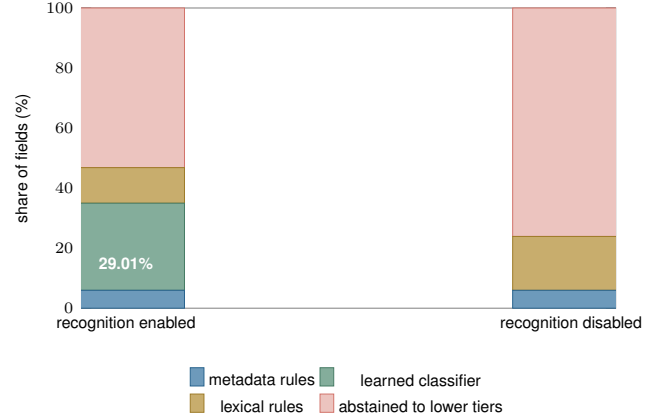


Fig. 4. How each field of the 131-service corpus is resolved, with and without the learned recognition tier. Takeaway: the classifier roughly doubles the share of fields bound to a semantic provider, from 22.93% to 44.79%.

TABLE II
CORPUS PROFILE OVER 131 SERVICE DEFINITIONS, TWO ROWS PER ENTITY. PERCENTAGES ARE SHARES OF ALL FIELDS IN THAT MODE. BOTH RUNS ISOLATE RECOGNITION, SO THE LANGUAGE MODEL IS GIVEN A BUDGET IT CANNOT USE; THE TIMES ARE THEREFORE THE RECOGNITION PATH, NOT THE WHOLE PIPELINE.

Measure	Recognition on	off
Services generating	131	119
Failures	0	12
Fields	11,529	9,928
bound to a provider	44.79%	22.93%
classifier	29.01%	—
metadata rules	5.96%	5.96%
lexical rules	11.85%	17.94%
abstained	53.18%	76.10%
Median service time	1,442 ms	107 ms

On 41 services and 21 domains the model never saw, it makes 1,206 routing decisions of which 1,160 are correct: 96.2% precision across 44 roles. Recall on supported fields — those whose expected role is one the head routes and which carry no decisive annotation — is 1,277/2,026 (63.0%). Status detection, the hardest single case because status columns rarely carry annotations, is 27/44 across 22 services and 10 domains. The critical false-positive rate — a role from a different family, or a role on a field reviewers marked unknown — is 2.2% of accepted decisions.

Two numbers in Table III belong together. Status recall is 27/44, while the classifier's top label is correct on 33/44. The first gap is the routing policy declining what it cannot justify; the second is the model being wrong. Only the second closes with more labels.

The residual errors concentrate on one boundary: a generic caption against a specific named thing. The model answers `description` where the panel said `currency_name`, `country_name`, or `org_name` — the same boundary on which the judges themselves needed a guideline revision.

TABLE III

SEALED EVALUATION ON 41 SERVICES NEVER USED FOR TRAINING OR CALIBRATION (4,317 FIELDS). PRECISION COUNTS THE CLASSIFIER’S OWN ACCEPTED DECISIONS; DETERMINISTIC METADATA AND LEXICAL DECISIONS ARE LISTED SEPARATELY BECAUSE THEY ARE RULES, NOT MODEL OUTPUT. TAKEAWAY: WHEN THE MODEL SPEAKS IT IS RIGHT 96.2% OF THE TIME.

Measure	Value
Classifier routing decisions	1,206
correct	1,160 (96.2%)
critical false positives	27 (2.2%)
Distinct roles routed	44
Supported-field recall	63.0%
Status fields detected	27/44
classifier’s top label correct	33/44
Decisions from metadata rules	389
Decisions from lexical rules	512

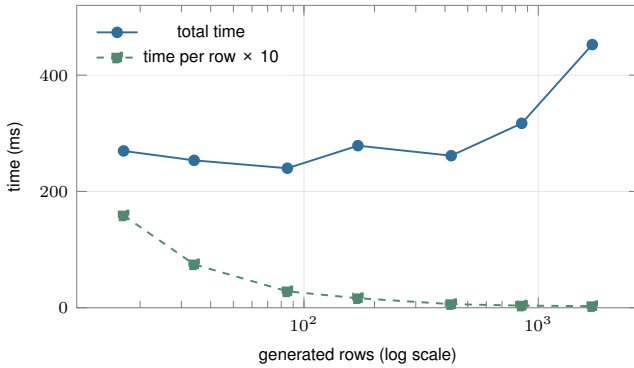


Fig. 5. Deterministic-pipeline scaling on a 17-entity service: total time rises from 270 ms to 453 ms while rows rise from 17 to 1,700, so cost per row falls by roughly sixty times. Takeaway: everything except the language model is effectively free at realistic dataset sizes.

C. Speed

Generation time is dominated by fixed costs, not dataset size (Fig. 5). On a 17-entity service with recognition disabled, 17 rows cost 270 ms and 1,700 rows cost 453 ms — from 15.9 ms per row down to 0.27 ms per row. Across the corpus the median service completes in 107 ms without recognition and 1,442 ms with it, the difference being model load and per-field inference; the 90th percentile with recognition is 2,929 ms and the whole corpus takes 251 s. Those two runs isolate recognition, so both give the language model a budget it cannot use. Letting it run at its packaged budget raises the median to 3,808 ms and the corpus to 820 s, with the same fields resolved by the same tiers — the extra time buys display text for columns no catalog and no role could fill. The reference application, with every tier enabled, generates in 2,845 ms.

D. End-to-end behaviour

A replay across four services covering both protocols completes in every mode the product supports. One case fails by design: deterministic mode refuses a service whose display domains are synthetic, because no verifier is available in that mode to check proposed captions.

The shipped build passes 466 package tests, 80 training-tool tests, and 50 editor tests with 0 lint errors, and produces

an extension of 11.4 MB (version 1.52.0) pinned to package 0.1.0-dev.10 at API version 2.

XV. A COMPONENT THAT DID NOT WORK

A. What it was for

Before a language-model-proposed display value is written, a verifier should judge whether it suits the field. The intended gate was that the verifier accepts at least 80% of valid candidates while accepting at most 1% of deliberately wrong ones.

B. What we measured

We built the corpus: 3,346 field-value pairs judged by the same panel, drawn from 19 approved value sources with 19 further sources held out. We tried the frozen encoder with a logistic head, a fine-tuned bi-encoder, and a cross-encoder. None separated the classes. At the 1% wrong-value budget the verifier accepts 1.3% of valid candidates; loosening the budget enough to be useful admits about 14% of clearly-wrong values.

Expanding the corpus from seven service groups to 19 made the task harder rather than easier, and the panel judged a sixth of our supposed “wrong” values as acceptable. Both point the same way: judging whether a plausible value belongs to an unseen service is not learnable from these features, and the task as posed was not well defined.

C. What we ship

We ship the strict operating point: 5/399 valid candidates accepted and 6/704 wrong ones (0.9%). In practice it rejects nearly everything proposed.

That made the generator refuse to produce data at all for applications with synthetic display domains, because the pipeline treated “unverified” as fatal. We changed it: when the verifier declines every candidate, generation keeps the deterministic values, marks the resource as unverified, and emits a diagnostic. The developer gets a complete dataset with honest labelling instead of an error. The guardrail for a *missing* verifier is unchanged, so deterministic mode still refuses synthetic domains by design.

We report this as a negative result rather than tuning a number. A verifier that accepts almost nothing is safe and nearly useless, and saying so is more useful to the next implementer.

XVI. DISCUSSION

A. Three mistakes, one lesson

The pilot recogniser routed nothing useful, and none of the three causes was the model. The input text was written for a machine rather than for the encoder. The head had too little capacity for the class count. The routing threshold came from a registry default that ignored the head’s own calibration, so calibrated decisions between 0.6 and 0.9 never routed.

Each was invisible in aggregate accuracy and obvious once every sealed decision recorded its confidence, its threshold, its prediction-set size, and its abstention reason. Instrumentation turned one mystery into three separate, fixable defects.

B. Machine labels buy scale, not truth

The panel produced 23,741 labels for the cost of compute, and that is why the system routes 44 roles rather than a handful. But the panel is also the ceiling: where judges disagree, the model cannot do better, and our residual errors sit exactly where inter-judge agreement was lowest.

C. Abstention as a feature

Because a lower tier always produces a valid value, abstention costs realism but never availability. That asymmetry justifies gates that would be too conservative in a conventional classifier, and it is why we report precision on the classifier's own decisions: the product question is not "how often is the model right?" but "when it speaks, can we trust it?"

XVII. LIMITATIONS AND THREATS TO VALIDITY

Label bias. Judges are language models; a blind spot shared by all three passes the majority vote unnoticed. We reduce but do not remove this by using two vendors and by recording the share of decisions resting on the column name alone.

Coverage measurement. Corpus numbers count fields bound to a semantic provider, not fields a human would judge realistic. They are a structural measure; the sealed evaluation is the quality measure.

Corpus scope. The corpus is 131 services parsable in the two supported formats, out of 473 registered; services stored only as derived graphs are excluded. The sealed set is family-disjoint but drawn from the same registry, so it measures generalisation across services rather than across ecosystems.

Recall. The head routes 63.0% of supported sealed fields; many columns still fall through to lower tiers.

The verifier. It is conservative to the point of near inactivity, and the relevance task as posed was not well defined.

Gate movement. Thresholds were revised once, with the package owner's approval, when the sealed set grew from 45 fields to 4,317. Revising a gate after seeing results risks fitting the gate to the model; both old and new values, the reason, and the approval are recorded in the release baseline so a reader can apply the original gate.

XVIII. FUTURE WORK

Three directions follow. More labelled examples on the name-versus-description boundary, where both panel and model are weakest. A verifier defined differently — checking value shape against a field's own observed domain rather than plausibility in the abstract. And a calibration split chosen so code-list services appear in the calibration partition, which would let several near-miss roles cross the support floor.

XIX. CONCLUSION

MockGen turns a service definition into a complete, coherent, deterministic mock dataset without a backend, a network, or a human labelling effort. The cascade makes empty screens structurally impossible; the recogniser makes the values worth looking at. On 131 real services it generates 11,529 fields with 0 failures and binds 44.79% of them to a semantic provider,

against 22.93% and 12 failures without recognition; on 41 unseen services its learned decisions are correct 96.2% of the time across 44 roles. The engineering lessons are transferable and unglamorous: write a model's input as language, size the head to the class count, let calibration set the thresholds, and instrument the evaluation so a failure tells you which of those is wrong.

GLOSSARY

API	Application programming interface.
BAS	SAP Business Application Studio, a browser-based development environment.
CDS	Core Data Services, SAP's schema definition language.
Conformant calibration	A calibration method producing a set of plausible classes at a target coverage level.
EDMX	The XML document format for OData service metadata.
Fleiss' kappa	Agreement measure for more than two raters, corrected for chance.
IR	Intermediate representation; here, the schema graph.
MLP	Multilayer perceptron, a network with a hidden layer.
OData	Open Data Protocol, used by SAP Fiori applications.
ONNX	Open Neural Network Exchange, the packaged model format.
SFT	Supervised fine-tuning; here, the local language model tier.
vCPU	Virtual central processing unit.
VSIX	The package format for a code editor extension.